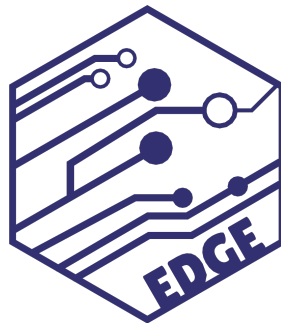




**Politecnico
di Torino**



Lab 1: Standalone Accelerator

RTL + Self-Checking Testbench

X-HEEP Accelerator Labs

A custom hardware accelerator design lab

Politecnico di Torino

EDGE Group

Objective

In Lab 0 you profiled the four applications and chose *one kernel* worth building custom hardware for. Lab 1 is where you build it — but *not* inside X-HEEP yet. You implement the accelerator as **standalone RTL**, and you prove it correct with a **self-checking testbench** that runs on *both* an open-source simulator (Verilator) and a commercial one (QuestaSim), checked against a *golden model* of your kernel.

The rule for this stage: no SoC, no bus fabric, no software driver. Just your accelerator, a testbench that feeds it inputs and checks its outputs, and the same numbers coming out of both simulators. Getting a kernel *correct* in isolation — and building the harness that tells you it is correct — is the foundation everything else (optimization in Lab 2, integration in Lab 3) sits on. This document describes what to do and the toolkit you are given to do it.

Worked example. The scaffold ships a complete testbench for **matmul** over an *OBI + CSR* interface. If you chose a different kernel or interface, you keep the same structure — golden vectors, transactors, self-checking testbench — and adapt the pieces. The parts you reuse unchanged (the CSR driver, the golden-vector checker, the Makefile flow) are the same for everyone.

1 The interface contract

Before any RTL, decide how your accelerator talks to the outside world. This is the one thing the testbench and your design must agree on. Two building blocks:

- **CSR / register interface** — a small set of memory-mapped *control/status/configuration* registers: start the accelerator, read back “done”, configure sizes and pointers. This is *control only*. Bulk matrix data does *not* travel through here.
- **OBI data port** — how the bulk data (the matrices) moves. You pick one of two styles:
 - **OBI master** (DMA-like): the accelerator itself reads A and B from memory and writes C back. The testbench provides the memory.
 - **OBI slave**: the testbench pushes A and B in and reads C out over the bus; the accelerator is a passive target.

The shipped example is **OBI master**. Its register map lives in `rtl/matmul_accel_pkg.sv` and is the contract the CSR driver relies on:

Offset	Name	Meaning
0x00	CTRL	bit 0 = start (write-1 pulse)
0x04	STATUS	bit 0 = done (RO), bit 1 = busy (RO)
0x08	N	matrix dimension (reset value 16)
0x0C	A_PTR	byte base address of A in memory
0x10	B_PTR	byte base address of B in memory
0x14	C_PTR	byte base address of C in memory

You are free to change this map — but if you do, update the CSR driver so the testbench still programs you correctly.

1.1 OBI: the bus underneath

Whichever side you pick, the data port speaks **OBI** (Open Bus Interface), the lightweight memory/peripheral bus the OpenHW Group specifies and X-HEEP uses throughout. The wire bundle in `tb/obi_if.sv` is not arbitrary — every signal is taken straight from the OBI specification. It is worth reading that spec (the PDF at <https://github.com/openhwgroup/obi>)

before you design the master FSM, so the handshake becomes a protocol you understand rather than a pattern you copy. The essential idea is a two-phase, *decoupled* handshake:

- **Address phase.** The master raises `req` with an `addr` (plus `we/be/wdata` for a write). The slave accepts it on the cycle it asserts `gnt`: a beat is committed exactly on `req && gnt`, and not before.
- **Response phase.** For a read, the data returns *later*, flagged by `rvalid` alongside `rdata` — possibly several cycles after the grant, and it is the slave, not the master, that decides when.

Because the two phases are decoupled, a compliant master may issue the next request while a previous read is still in flight. That pipelining is where an OBI master earns its bandwidth — and it is exactly the kind of optimization Lab 2 rewards. For Lab 1 a simple one-outstanding-request FSM (issue, wait for grant, wait for `rvalid`, repeat) is correct and sufficient. The `obi_mem_model` answers with a *configurable* read latency (RLAT) precisely so you can prove your handshake survives a slave that does not respond in a single cycle; bump it and your FSM must still work. Read the specification’s channel and handshake sections for the full rules (byte enables, error responses, and the optional signals X-HEEP does not use).

2 Vending and reusing IP

You already ran `make vendor` in Lab 0, but it is worth understanding what it actually does, because you lean on it again here and in Lab 3. *Vending* takes a snapshot of an upstream repository — pinned to an exact revision — and copies it into your tree, optionally applying local patches. It is *not* a git submodule: there is no live link to the remote, nothing to `git submodule update`, and the build stays reproducible and offline-buildable forever, because the exact bytes you built against live with your project. The descriptor is a small `.vendor.hjson` naming the URL, the `rev`, and any files to exclude or patches to apply; `util/vendor.py` reads it and performs the copy. Pinning the `rev` is what keeps “it built last year” true today.

The payoff for an accelerator designer is *reuse*. You do not build every FIFO, counter, and clock gate from scratch — and you should not: the soundest design reuses a proven, verified block instead of re-deriving one at 2am. X-HEEP already vendors two libraries of exactly the sub-blocks an accelerator needs, and because they sit in the vendored tree you can depend on them directly:

Library (VLNV)	What is in it
<code>pulp-platform.org::common_cells</code>	Reusable synthesizable RTL: FIFOs (<code>fifo.v3</code>), counters, synchronizers, arbiters and round-robin, clock-domain-crossing primitives, one-hot/gray coders, register slices. The parts you would otherwise re-implement and re-verify.
<code>pulp-platform.org::tech_cells_generic</code>	Technology-agnostic wrappers for cells that are otherwise process-specific: clock gates (<code>tc_clk_gating</code>), muxes, and SRAM macros (<code>tc_sram</code>). Behavioural in simulation, swapped for real standard/macro cells at synthesis.

Two teaching points. First, *reach for these before hand-rolling*: if your datapath wants an input FIFO to decouple the OBI reads from the MAC pipeline, `fifo.v3` already exists and is one instantiation away. Second, `tech_cells_generic` is what keeps your RTL *portable*: by instantiating `tc_clk_gating` or `tc_sram` rather than a vendor-specific primitive, the same source elaborates in Verilator, maps in the open-source synthesis flow, and maps again in the commercial one — the two flows you must compare in Lab 2. Hard-coding a foundry cell would break one of them.

Making a library available is one line in your core file: add its VLVN to the `depend:` list of the fileset that needs it, and FuseSoC resolves it from the vendored X-HEEP.

```
filesets:
  rtl:
    depend:
      - pulp-platform.org:common_cells
      - pulp-platform.org:tech_cells_generic
    files:
      - rtl/matmul_accel.sv
    file_type: systemVerilogSource
```

The upstream sources and their documentation live at https://github.com/pulp-platform/common_cells and https://github.com/pulp-platform/tech_cells_generic.

3 Generating the register file with reggen

The CSR block from the interface contract — the address decode, the read multiplexer, the per-bit write enables, the read-only versus read-write versus write-one-to-pulse semantics — is mechanical, repetitive, and a classic place for hardware and software to drift apart: you add a field in the RTL, forget the matching C `#define`, and the driver writes the wrong bit. The disciplined way to build it is to *not* hand-write it at all. You describe the registers *once* in a small HJSON file and a generator emits both the RTL and a matching C header from that single source. Change the description, regenerate, and the two can no longer disagree.

Use the X-HEEP version, not the mainstream one. This is the one warning to internalize. The tool is OpenTitan’s `regtool` (commonly called “reggen”), but X-HEEP ships its *own vendored and patched copy*, wrapped as a FuseSoC generator named `regtool` (VLVN `xheep:util:reg-generator`). Do *not* `pip install reggen` and run the stock tool: the mainstream version may accept slightly different HJSON keys and, more importantly, emit a different register-bus flavour than the `reg_interface` that X-HEEP expects. Always drive the copy that came with your vendored `x-heep/`. The authoritative how-to is `x-heep/docs/source/How_to/RegGenerator.m` the upstream reference for field semantics — what `swaccess` and `hwaccess` mean, the full register and field grammar — is the OpenTitan manual at <https://opentitan.org/book/util/reggen/index.html>. Read the manual to understand the fields, but defer to the X-HEEP wrapper for how you invoke it.

3.1 The register description

Your registers live in `data/matmul_accel.hjson`. It declares a `reg_iface` device interface, a 32-bit register width, and then one entry per register, each with its software access mode (`swaccess`), hardware access mode (`hwaccess`), reset value, and fields. A trimmed version of the `matmul_accel` block:

```
{ name: "matmul_accel"
  bus_interfaces: [ { protocol: "reg_iface", direction: "device" } ]
  regwidth: "32"
  registers: [
    { name: "CTRL", swaccess: "wo", hwaccess: "hro",
      desc: "Control: bit 0 = start (write-1 pulse)",
      fields: [ { bits: "0", name: "START" } ] }
    { name: "STATUS", swaccess: "ro", hwaccess: "hwo",
      desc: "Status: done, busy",
      fields: [ { bits: "0", name: "DONE" }, { bits: "1", name: "BUSY" } ] }
    { name: "N", swaccess: "rw", hwaccess: "hro", resval: "16",
```

```

    fields: [ { bits: "31:0", name: "N" } ] }
  { name: "A_PTR", swaccess: "rw", hwaccess: "hro",
    fields: [ { bits: "31:0", name: "A_PTR" } ] }
  // ... B_PTR, C_PTR likewise
]
}

```

The `swaccess/hwaccess` pair is the whole point: `CTRL.START` is `wo` from software and `hro` (hardware-read-only) — software pulses it, hardware reads it — while `STATUS` is `ro` from software and `hwo` (hardware-write-only), because hardware produces `done/busy` and software only observes them. Specify the modes correctly and the generator builds exactly the right flops and enables for you.

3.2 What it produces, and how the accelerator uses it

Running the generator over `data/matmul_accel.hjson` produces three artefacts:

Artefact	Role
<code>rtl/matmul_accel_reg_pkg.sv</code>	The generated package: the <code>reg2hw</code> and <code>hw2reg</code> struct types that carry each field between the CSR block and your logic.
<code>rtl/matmul_accel_reg_top.sv</code>	The CSR block itself: a <code>reg_req</code> in / <code>reg_rsp</code> out register interface on one side, and the <code>reg2hw/hw2reg</code> structs on the other. All the decode and read-mux you did not have to write.
<code>sw/matmul_accel_regs.h</code>	The C header of register offsets and field masks the Lab 3 driver will <code>#include</code> — generated from the <i>same</i> HJSON, so it cannot drift from the RTL.

You then instantiate `matmul_accel_reg_top` inside your accelerator and wire it up: give it the CSR bus — the very same register interface the testbench's `reg_driver` drives — and it hands you `reg2hw` (read `reg2hw.n`, `reg2hw.a_ptr`, the `reg2hw.ctrl.start` pulse) and accepts `hw2reg` (drive `hw2reg.status.done` and `.busy` from your FSM). This is precisely why the register map in `rtl/matmul_accel_pkg.sv` and the HJSON must describe the *same* layout: the testbench programs offsets defined in the package, and your hardware decodes them in the generated block.

3.3 The FuseSoC generator

You never call the tool by hand. In FuseSoC a *generator* is a build-time program that *produces source before elaboration* — here, HJSON in, SystemVerilog and a C header out — declared in your `.core` file. You add `xheep:util:reg-generator` as a dependency and a `generate:` block that names the `regtool` generator and its parameters:

```

generate:
  matmul-accel-regs:
    generator: regtool
    parameters:
      name: matmul_accel
      config: data/matmul_accel.hjson
      rtl_dir: rtl
      sw_path: sw/matmul_accel_regs.h
      doc_path: sw/matmul_accel_regs.md

targets:
  default:
    filesets: [rtl]

```

```
generate: [matmul-accel-regs]
```

All paths are relative to the directory of the `.core` file. Two commands drive it:

```
make vendor # pull X-HEEP into ./x-heap -- brings the reggen generator + IPs
make regs # run the regtool generator over data/matmul_accel.hjson
```

Run these inside the `core-v-mini-mcu` conda environment, the one that also provides `fusesoc`. This is not a bespoke mechanism: it is exactly how X-HEEP generates the registers for *all* of its own peripherals — `soc_ctrl`, `pad_control`, and the rest — and it is the same `.core` and the same generator you carry into the SoC in Lab 3. Building the register block this way now means it is already in the shape integration expects.

4 The golden model

A self-checking testbench needs a *reference*: the correct outputs to compare against. You generate them two ways, and they must agree.

1. **Python golden** (`model/golden.py`) — reproduces the kernel and writes `vectors/a.hex`, `b.hex`, `c_golden.hex` (one 32-bit word per line, row-major), the format the testbench loads with `$readmemh`.
2. **C dump from the real kernel** (`make host-vectors`) — compiles the *actual* Lab-0 C kernel for your host machine and dumps the buffers it reads and writes, using the `DUMP_HEX(...)` macros in `model/dump.h`. On the X-HEEP target those macros compile to nothing, so the same C runs unchanged in Lab 3.

The second route is the important one: it is how you obtain a golden for *whatever kernel you actually picked*, without re-implementing the algorithm in a second language. Wrap the region you accelerate:

```
#include "dump.h"
...
DUMP_HEX("vectors/a.hex", &a[0][0], N*N); // inputs the kernel reads
DUMP_HEX("vectors/b.hex", &b[0][0], N*N);
kernel(); // the code you accelerate
DUMP_HEX("vectors/c_golden.hex", &c[0][0], N*N); // outputs it produced
```

For `matmul` both routes produce identical files (same checksum) — that cross-check is deliberate: if they disagree, one of your two references is wrong before you have written a line of RTL.

5 The testbench toolkit

You do not write the testbench from scratch. You *mount* one from a small library of components, picking the transactors that match your interface. Each lives in its own file under `tb/`.

File	Role	Use when
<code>obi_mem_model.sv</code>	OBI <i>slave</i> RAM (backdoor load/read)	accel is OBI <i>master</i>
<code>obi_master_bfm.sv</code>	OBI <i>master</i> driver (<code>write/read_word</code>)	accel is OBI <i>slave</i>
<code>reg_driver.sv</code>	CSR write / read / poll	always
<code>vector_check.sv</code>	load golden, compare, report (in the TB)	always
<code>obi_if.sv, reg_if.sv</code>	interface wire bundles	always
<code>tb_matmul_accel.sv</code>	top testbench (wires the OBI-master case)	the file you edit

The two OBI components are mirror images — you instantiate exactly *one*:

- Accelerator is an **OBI master**? It drives the bus; the testbench must *answer*. Instantiate `obi_mem_model` — a behavioural RAM that grants requests and returns data. The testbench backdoor-loads A and B into it and reads C back out of it.
- Accelerator is an **OBI slave**? It waits for the bus; the testbench must *drive*. Instantiate `obi_master_bfm` and call `write_word/read_word` to push A/B in and read C out.

The `reg_driver` and `vector_check` are used unchanged either way. The top testbench then reads like the description of the test itself:

```
// preload inputs into the RAM (backdoor)
u_mem.load_hex(A_BASE, {vdir, "a.hex"}, WORDS);
u_mem.load_hex(B_BASE, {vdir, "b.hex"}, WORDS);

// program the accelerator over CSRs
u_regdrv.write(REG_N, N);
u_regdrv.write(REG_A_PTR, A_BASE);
u_regdrv.write(REG_B_PTR, B_BASE);
u_regdrv.write(REG_C_PTR, C_BASE);
u_regdrv.write(REG_CTRL, 1 << CTRL_START_BIT); // GO

// wait for completion, then compare C against the golden vector
u_regdrv.poll_bit(REG_STATUS, STATUS_DONE_BIT, TIMEOUT, ok);
for (int i = 0; i < WORDS; i++) got[i] = u_mem.word(C_BASE + 4*i);
errors = compare(got, {vdir, "c_golden.hex"}, WORDS);
```

The comparison happens *inside* the testbench: `vector_check` loads `c_golden.hex`, diffs it against what the accelerator produced, prints the first mismatches, and the testbench exits `TEST PASSED` (status 0) or `TEST FAILED` (non-zero). No external diff step.

5.1 One testbench, both simulators — and why not DPI

The entire testbench is plain SystemVerilog. It compiles on *both* Verilator 5 (`--binary --timing`) and QuestaSim, so nothing is written twice: the transactors are the single source of truth for both tools. A DPI bridge (calling a C golden at simulation time) would also avoid duplication, but here it buys nothing over precomputed `$readmemh` vectors, so the scaffold does not use it. If a future kernel needs a golden computed on the fly, DPI is the hook you would add — but you do not need it for Lab 1.

6 What you implement

Exactly one file: `rtl/matmul_accel.sv`. It ships as a stub that elaborates but computes nothing (so the testbench correctly reports a mismatch until you fill it in). For the reference OBI-master contract you build:

1. **CSR block** — decode `reg_addr_i`; hold N, A_PTR, B_PTR, C_PTR; latch the START pulse; expose `STATUS.{done,busy}`; always assert `reg_ready_o`.
2. **FSM + datapath** — on START, stream A and B from memory over the OBI master port, compute $C = A \cdot B$ in `int32`, write C back, then raise `STATUS.done`.
3. **OBI master handshake** — assert `req/addr (+we/be/wdata for writes)`; a beat is accepted on `gnt`; read data returns later on `rvalid/rdata`.

Build the *naive* version first — one multiply-accumulate, the plain triple loop. Correctness now; the optimization (pipelining, folding, a MAC array) is Lab 2's job, and it will be measured *against* this baseline.

7 Commands

Run everything from the `lab1/` directory.

```
make vendor # pull X-HEEP into ./x-heap (reggen generator + IPs)
make regs # generate the CSR block from data/matmul_accel.hjson

make vectors # golden vectors from the Python model
make host-vectors # the SAME vectors, dumped from the real C kernel

make sim # Verilator 5: build + self-checking run
make questa # QuestaSim / ModelSim: same testbench, same result
make sim-fusesoc # same run through the FuseSoC .core (a Lab 3 preview)

make waves # Verilator run with FST tracing (then: gtkwave dump.fst)
make clean
make help
```

8 Reading the output

A run ends in exactly one of two lines. Passing:

```
$ make sim
[tb] vectors from 'vectors/', N=16 (256 words/matrix)
TEST PASSED: 256/256 words match
```

Failing (the shipped stub, until you implement the datapath):

```
$ make sim
[tb] vectors from 'vectors/', N=16 (256 words/matrix)
  MISMATCH [0]: got 00000000 exp 000004b0
  ...
TEST FAILED: 256/256 words mismatch
```

If the accelerator never sets `STATUS.done`, the CSR poll times out instead (`TEST FAILED: STATUS.done not set within ... cycles`) — that means your FSM never finished (or never started). Use `make waves` to see whether the OBI/CSR activity matches what you expect; waveforms are a **debugging aid only** here, not a graded artifact.

Both simulators must give the *same* result. That agreement — open-source and commercial tool, one testbench — is itself a checked outcome.

9 FuseSoC

`matmul_accel.core` drives the same sources through FuseSoC (`make sim-fusesoc`). Read it: it is the same `.core` file format you will *extend* in Lab 3 to pull the accelerator into X-HEEP. Understanding its `filesets` and `targets` now makes that step routine later.

Deliverables

- `rtl/matmul_accel.sv` implemented; `make sim` prints `TEST PASSED`.
- `make questa` prints the *same* `TEST PASSED` (upload the transcript — the commercial path is checked from your artifacts).
- Both golden routes (`make vectors`, `make host-vectors`) produce identical vectors; note the shared checksum in your report.
- A short note: OBI master vs. slave — which you chose and why, and how the two transactors differ.



The second half of Lab 1 — baseline synthesis and PPA (Yosys and `dc_shell`) — follows in the synthesis guide, once your RTL passes simulation here.